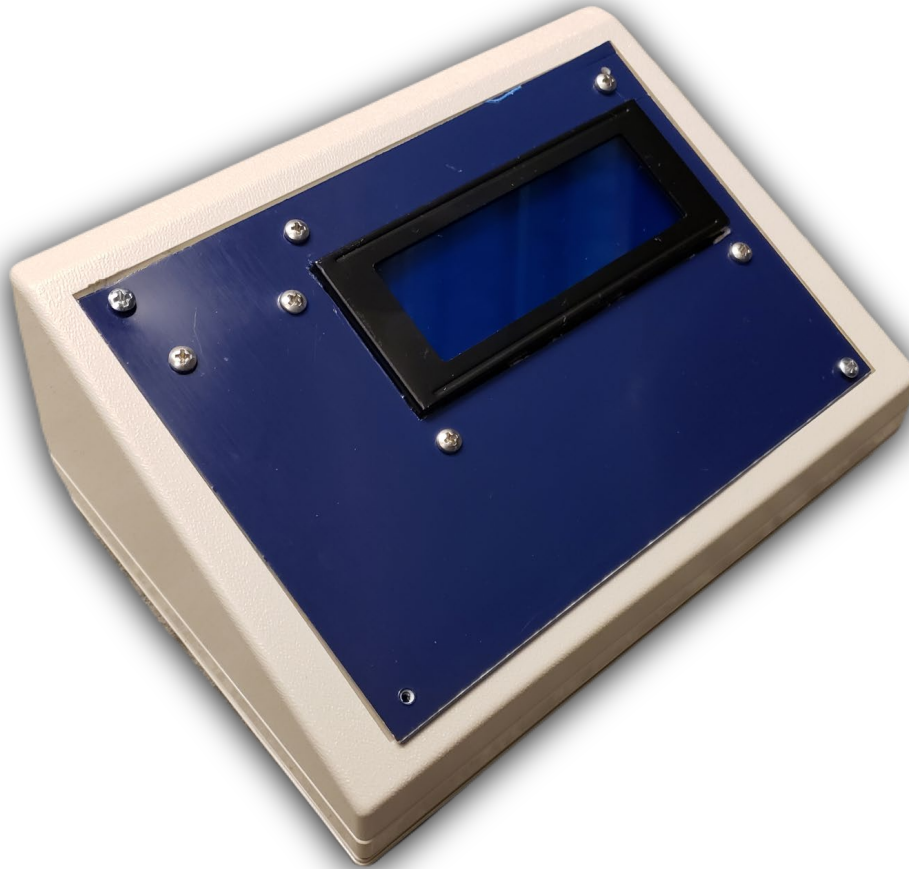


# **RFID Punch Clock System**

## **Final Report**



Rares Covaci  
Jeremy E Edwards  
Jesse Owens  
Blake Truong

# Abstract

This project aimed to develop an Arduino Mega-based punch clock system, integrating various modules including the Adafruit Ultimate GPS+Logging Shield, an RFID reader (MFRC522), an I2C LCD 20x4 display, and an SPI SD card. The system was designed to record the time and date of employee or student check-ins using RFID technology, with the time data synchronized through GPS. The primary challenge was ensuring accurate timekeeping, which involved converting UTC to PST, including adjustments for daylight savings and leap years. Additional challenges included resolving GPS data parsing issues when the RFID reader was connected and ensuring the time display was correct upon device power-up. These were addressed through software optimizations, such as implementing non-blocking RFID checks and modifying the code to display time independent of GPS fixes. The final system effectively demonstrates a seamless integration of various technologies to create a functional and reliable punch clock system. This report details the project's design, implementation, challenges, and solutions, highlighting the successful application of embedded systems in solving real-world problems.

## Introduction

### Background

Punch clock systems are an essential component of workforce management, enabling the accurate tracking of employee attendance and working hours. Traditionally, these systems have evolved from mechanical time clocks to more sophisticated digital solutions, incorporating a variety of technologies to enhance efficiency and precision. The emergence of microcontroller platforms, such as Arduino, has opened up new possibilities in the realm of custom-built time and attendance systems. These platforms offer a blend of flexibility and functionality, allowing for the integration of diverse sensors and modules to meet specific project needs.

### Project Objective

The primary objective of this project was to design and implement a punch clock system using the Arduino Mega as the core processing unit. The Arduino Mega was specifically chosen for its extensive number of supporting pins, which are essential for accommodating the multiple components required in our project. The system integrates an Adafruit Ultimate GPS+Logging Shield, which is pivotal for its real-time clock (RTC) and GPS functionality, an RFID reader (MFRC522) for employee identification, an I2C LCD 20x4 display for real-time data presentation, and an SPI SD card module for data logging. This project aimed to develop a reliable, efficient, and cost-effective solution for monitoring and recording employee attendance with precision.

### Significance

This punch clock system is particularly significant for its application in small to medium-sized enterprises (SMEs), which often seek economical yet effective solutions for attendance tracking. By harnessing the power of the Arduino platform and its compatible modules, this project demonstrates the potential of technology to streamline and automate key business processes. Moreover, it serves as an excellent educational tool, showcasing the practical application of embedded systems and software integration in addressing real-world challenges.

# Materials and Methods

## Hardware Components

The project utilized the following key hardware components, chosen for their specific functionalities and compatibility with the Arduino platform:

1. **Arduino Mega 3560 R3:** The central processing unit for the system, selected for its substantial number of I/O pins, essential for our project's complex requirements.

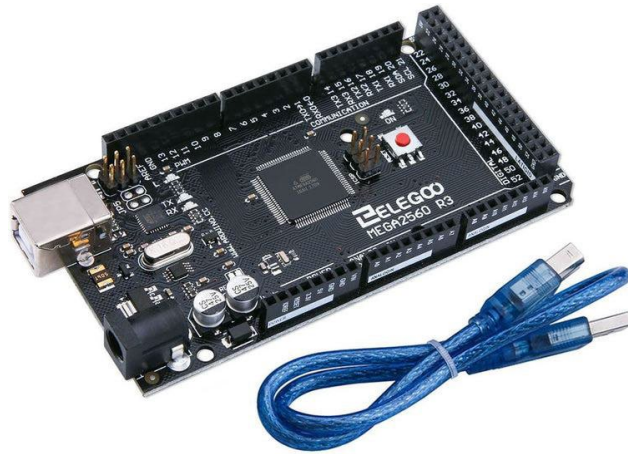


Figure 1. MEGA 2560 R3 Board with USB Cable

2. **Adafruit Ultimate GPS+Logging Shield:** Provides GPS functionality and time synchronization, and includes a built-in Real-Time Clock (RTC) and an SD card logger for data storage. The module maintains accurate timekeeping even when the GPS is unavailable, ensuring continuous accuracy.

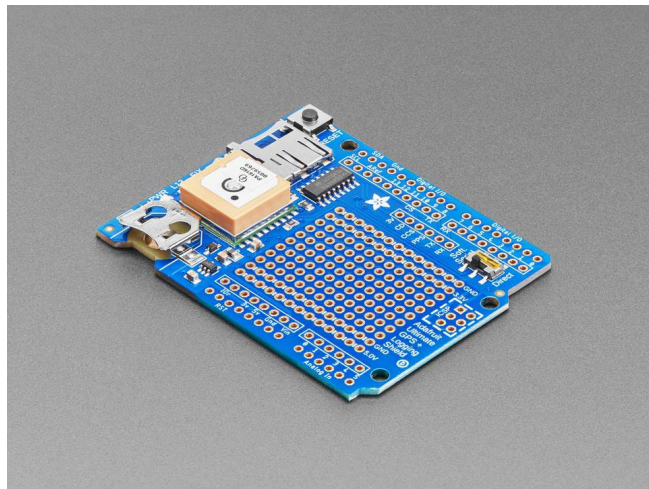
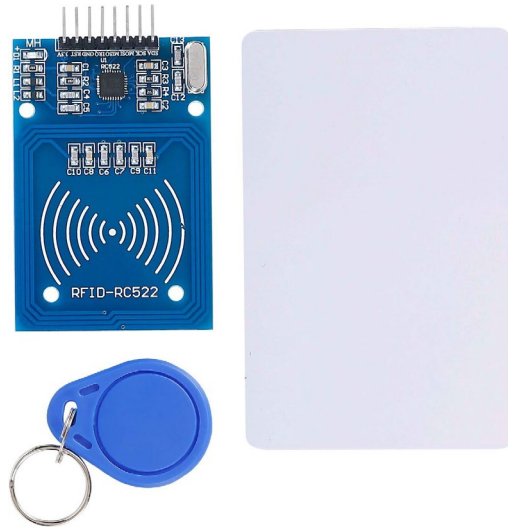


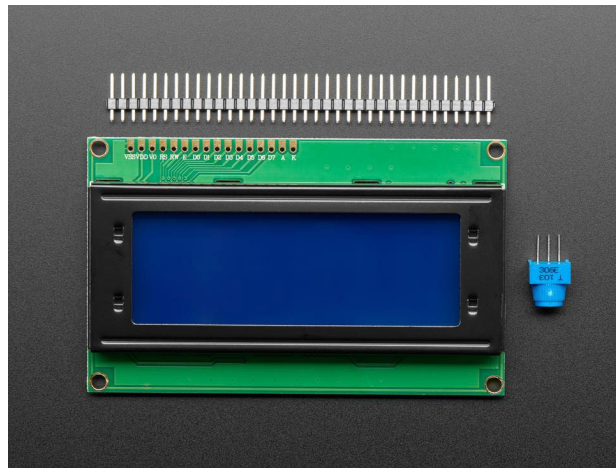
Figure 2. Arduino Ultimate GPS Shield

3. **RFID Reader (MFRC522):** Employed for scanning RFID tags for employee identification. It operates at a frequency of 13.56 MHz and is capable of reading unique User Identification (UID) tags.



**Figure 3. RFID Sensor**

4. **I2C LCD 20x4 Display:** Used for displaying real-time data, including date, time, and RFID scan results. The choice of a 20x4 LCD was influenced by its capacity to display sufficient information, interfacing through I2C connections for efficient integration.



**Figure 4. LCD 20x4**

5. **SPI SD Card Module:** Integrated as part of the GPS Shield for logging data. This component allows for efficient recording of user check-ins and time stamps directly onto an SD card.

## Circuit Design

Each module was connected to the Arduino Mega as follows:

- **GPS Shield:** Connected to Serial1 ports with GPS TX to Arduino RX1 and GPS RX to Arduino TX1.
- **RFID Reader:** Utilized the SPI interface for communication, with specific pins allocated for SDA, SCK, MOSI, MISO, RST, GND, and 3.3V.

- **LCD Display:** Interfaced with the Arduino Mega primarily through I2C connections, facilitated by an I2C adapter.
- **SD Card Module:** Connected through dedicated SPI pins on the Arduino, as part of the GPS Shield.

A sketch of the connections between the GPS shield, LCD, and RFID is shown in figure 5 below. The GPS logger shield schematic is also shown in figure 6.

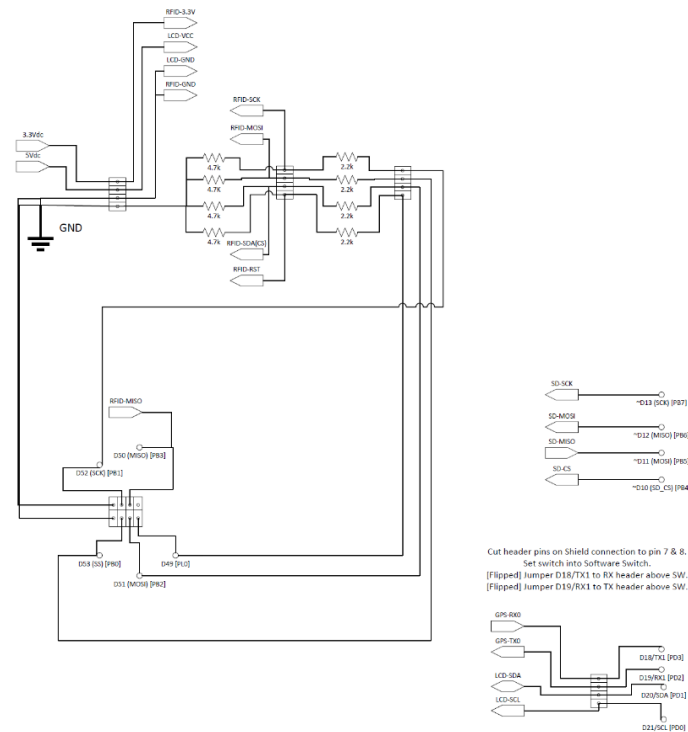


Figure 5. Shield Connection with LCD and RFID (Click on image to see it larger)

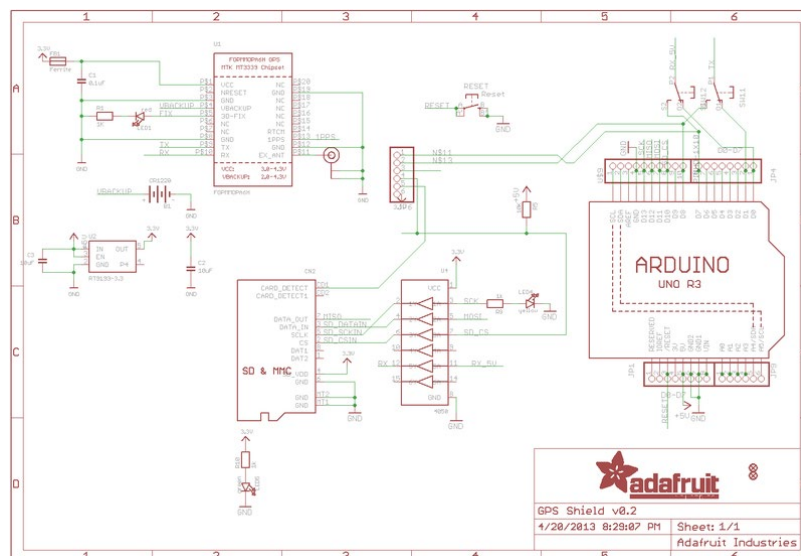


Figure 6. Ultimate GPS logger shield schematic (Click on image to see it larger)

## Software Implementation

The software development process involved several key steps:

- **Module Testing:** Each component (GPS, RFID, LCD, SD Card) was individually tested with the Arduino Mega to ensure proper functionality. This included using example codes from relevant libraries for each component.
- **Integration of LCD and GPS Module:** The code was initially written to display the current date and time from the GPS on the LCD.
- **RFID Module Integration:** Subsequently, the RFID module was added, with modifications made to the code to ensure compatibility and simultaneous functioning with the GPS and LCD modules.
- **SD Card Functionality:** The final step involved integrating the SD card module, enabling the system to log each RFID scan with the corresponding date and time.

## Software Libraries

Various Arduino libraries were utilized, including:

- **Adafruit\_GPS.h** for GPS module functionality.
- **Wire.h** and **LiquidCrystal\_I2C.h** for the I2C LCD display.
- **SPI.h** and **MFRC522.h** for the RFID reader.
- **SD.h** for SD card operations.

## Data Handling

The system was programmed to:

- Synchronize GPS time to Pacific Standard Time (PST) considering leap years (but NOT daylight savings).
- Process RFID data by scanning employee/student tags and transmitting UID data to the Arduino.
- Manage the display to show real-time information including the time, date, and RFID scan details.
- Log each RFID scan event with date and time onto the SD card in a text file format.

# Challenges and Solutions

## Challenge #1: Time Conversion

- **Problem:** The GPS module provided time in UTC, necessitating conversion to Pacific Standard Time (PST). This involved handling the 8-hour time difference, including adjustments for day, month, and year transitions, and accounting for leap years.
- **Solution:** Implementation of the `adjustForPST` function, which adjusted the time from UTC to PST, ensuring correct handling of the end of months and leap years.

## Challenge #2: GPS Data Parsing with RFID

- **Problem:** Initial integration of the RFID module with the GPS module led to issues in GPS data

- parsing, suspected to be caused by the blocking nature of RFID checks.
- **Solution:** Modification of the RFID reading process to a non-blocking approach, ensuring continuous GPS data reading and resolving the parsing issues.

#### Challenge #3: GPS Fix Delay and Time Display

- **Problem:** On powering the device, the time display was incorrect until a GPS fix was obtained, causing delays of 15-20 minutes for accurate time tracking.
- **Solution:** Code was updated to process and display time data regardless of GPS fix status, ensuring the display of time as soon as any NMEA data was received from the GPS module.

#### Challenge #4: Integration of GPS Functionality with RFID System

- **Problem:** Encountered difficulties in integrating the GPS functionality with the RFID system, largely due to the complexity of the Ultimate GPS Logger Shield and its interactions with different modules
- **Solution:** Persistent efforts by team members to resolve these integration challenges led to the successful combination of the GPS and RFID components, streamlining the rest of the system integration.

#### Challenge #5: Code Optimization and System Performance

- **Problem:** As development progressed, the need for code optimization and enhancement of system performance became evident.
- **Solution:** Various updates were made in subsequent version updates, including refining the LCD display refresh logic, adding leading zeroes to date and time for consistent formatting, and rearranging operational sequences for improved performance and user experience.

## Results and Discussion

The final iteration of the Arduino-based punch clock system demonstrated successful integration of various components, highlighting its robust functionality. This section discusses the key outcomes and insights from the project. A link to a video showcasing the functionality of our project is also included at the end of this section.

### Functionality and Performance

The system efficiently identified RFID tags, displaying the UID on the LCD and logging the data onto the SD card. Concurrently, the GPS module provided accurate time data in Pacific Standard Time, essential for the punch clock's functionality. This precise timekeeping was crucial for the system's effectiveness. The project also excelled in ensuring seamless communication between individual components. The SD card and RFID reader interacted smoothly through the Arduino's SPI pins, and the 20x4 LCD effectively displayed data via the I2C protocol. The built-in Real-Time Clock in the GPS module, continuously synchronized time, ensuring consistent accuracy in timekeeping.

### Testing and Performance Analysis

In the initial testing phase, each component, including the SD card, GPS, and RFID reader, was



methodically tested using example codes from their respective libraries. This phase was crucial for understanding how each component functioned and how they could be integrated effectively. Despite minor delays in the LCD refresh during simultaneous RFID scans and GPS data updates, the overall system performance remained robust. Overcoming the integration challenges, particularly in combining the GPS functionality with the RFID system, marked a significant achievement in the project's development.

### Insights and Learning Outcomes

The project provided valuable insights into embedded system design, especially in the context of integrating multiple technologies and understanding the subtleties of software-hardware interactions. It highlighted the importance of a systematic approach to problem-solving, particularly vital in managing real-time data processing and coordinating multiple modules. Successfully integrating the diverse components of the system not only showcased the versatility of the Arduino platform in handling complex tasks but also its applicability in creating practical solutions for real-world problems.

### Potential Improvements and Future Work

Looking forward, the system offers opportunities for further enhancements. Adding a more interactive user interface, such as buttons or a touchscreen, could improve the user experience significantly. Implementing connectivity options like Wi-Fi or Bluetooth would enable remote data access and integration with cloud-based systems. While the GPS module's built-in battery maintains timekeeping, exploring other aspects of power efficiency could still be beneficial, particularly for battery-operated setups.

### Video and Picture Demonstration

To further illustrate the functionality and effectiveness of our Arduino-based punch clock system, we have included a video link showcasing the system in action. This video provides a real-time demonstration of the system's operation. Figure 7 represents our finished system before it was placed in its own enclosure.

Link: [https://drive.google.com/file/d/1x\\_P2JwyWwcXPPE55z\\_2R5GFPwomrykBW/view?usp=sharing](https://drive.google.com/file/d/1x_P2JwyWwcXPPE55z_2R5GFPwomrykBW/view?usp=sharing)

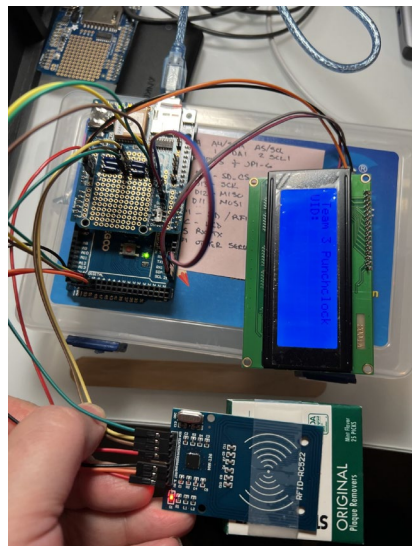


Figure 7. Bare Punch Clock System (Click on image to see it larger)



# Conclusion

The completion of the Arduino-based punch clock project marks a significant milestone in our abilities in the field of embedded systems, combining multiple technologies into a coherent and functional system. This project not only met its core objective of accurately tracking and recording time and attendance data but also provided an expansive learning experience in electronics, programming, and system integration.

The project's success was underpinned by effective communication with individual components such as the SD card, RFID reader, and the LCD display, as well as the precise timekeeping ensured by the GPS module's Real-Time Clock. One of the main key achievements was the successful integration of the GPS functionality with the RFID system, despite initial challenges. This integration proved to be one of the more complex aspects of the project, requiring a deep understanding of the GPS module's interaction with other components. Overcoming this hurdle was a testament to the team's problem-solving capabilities and persistence.

Looking ahead, the project lays a foundation for further improvements. Enhancing the user interface, perhaps through the addition of interactive elements like buttons or a touchscreen, and incorporating connectivity features for remote data access, are potential areas for development. While the GPS module's built-in battery efficiently maintains timekeeping, exploring other power efficiency aspects could prove beneficial for the system, especially in battery-dependent setups. In conclusion, the Arduino-based punch clock system stands as an exemplary demonstration of how technology can be harnessed to create efficient, cost-effective solutions for everyday challenges.

# References

## Websites

1. Real Time Clock Project - Arduino Project Hub: <https://projecthub.arduino.cc/ejshea/real-time-clock-d01fa3>
2. RFID Based Smart Attendance System - Arduino Project Hub: [https://projecthub.arduino.cc/team\\_chkr/rfid-based-smart-attendance-system-beff99](https://projecthub.arduino.cc/team_chkr/rfid-based-smart-attendance-system-beff99)
3. Interfacing RFID RC522 with Arduino MEGA - Instructables: <https://www.instructables.com/Interfacing-RFID-RC522-With-Arduino-MEGA-a-Simple-/>
4. Adafruit Ultimate GPS Logger Shield - Overview: <https://learn.adafruit.com/adafruit-ultimate-gps-logger-shield/overview>

## Libraries

1. Adafruit GPS Library: [https://github.com/adafruit/Adafruit\\_GPS](https://github.com/adafruit/Adafruit_GPS)
2. SD Library - Arduino Reference: <https://www.arduino.cc/reference/en/libraries/sd/>
3. LiquidCrystal Library - Arduino Reference: <https://www.arduino.cc/reference/en/libraries/liquidcrystal/>
4. MFRC522: <https://github.com/miguelbalboa/rfid>

# Appendix

## Code Listing

```
GPS_RFID_LCD_SD.ino
/*
-----
GPS_RFID_LCD_SD Updates:
- Built off GPS_RFID_LCD_v3 added SD Card writeSDCard(String sData) and readSDCard().
- Changed RFID RST → FROM: Arduino Digital Pin 5 TO: 49 as per Project Schematic.
-----
v3 Updates:
- Added spaces after date and time so residual characters will not show.
- The time data (hour, day, month, year) is now always updated when new NMEA data is
  received and parsed, regardless
  of whether the GPS has a fix. This change allows the time data to be displayed on the
  LCD as soon as any NMEA data is available.
-----
v2 Updates:
- Improved code to remove the apparent refresh on LCD.
- Added leading zeroes to date and time (so there's no need for refresh). Date now shows
  MM/DD/YY instead of DD/MM/YY (sorry about that).
-----
Arduino-Based GPS and RFID Tracking System with LCD Display. No Wifi or card reader
(yet).
NOTE: This code will work okay and displays the right info on the LCD if the GPS has a
fix to the satellite.
I am thinking it should also work if battery is able to keep time and date updated
without GPS connection.
Connections:
*GPS Shield Module (Connected to Serial1):
  - GPS TX → Arduino RX1
  - GPS RX → Arduino TX1
*RFID Reader (MFRC522, using SPI):
  - RFID SDA (SS) → Arduino Digital Pin 53 (SS Pin for SPI)
  - RFID SCK → Arduino Digital Pin 52 (SCK Pin for SPI)
  - RFID MOSI → Arduino Digital Pin 51 (MOSI Pin for SPI)
  - RFID MISO → Arduino Digital Pin 50 (MISO Pin for SPI)
  - RFID IRQ → No connection
  - RFID RST → Arduino Digital Pin 49
  - RFID GND → Arduino GND
  - RFID 3.3V → Arduino 3.3V
*I2C LCD Display:
  - LCD SDA → Arduino SDA
  - LCD SCL → Arduino SCL
  - LCD VCC → Arduino 5V
  - LCD GND → Arduino GND
*SPI SD Card on GPS Shield Module:
  - SD SCK → Arduino Digital Pin 13
  - SD MOSI → Arduino Digital Pin 12
  - SD MISO → Arduino Digital Pin 11
```

- SD CS (SS) → Arduino Digital Pin 10

```
-----
*/

#include <Adafruit_GPS.h>           // Library for GPS module
#include <Wire.h>                   // Library for I2C communication
#include <LiquidCrystal_I2C.h>      // Library for I2C LCD display
#include <SPI.h>                    // SPI library for RFID reader
#include <MFRC522.h>                // Library for RFID module
#include <SD.h>                     // Library for SD Card on adafruit GPS module

#define RST_PIN 49 // RFID Reset Pin
#define SS_PIN 53 // RFID SS Pin
#define LOG_FILE "Punch.txt" // Log file to store punchclock data
const int chipSelect = 10; // SD CS (SS) Pin
File myFile; // LOG_FILE File on SD for punchclock data

Adafruit_GPS GPS(&Serial1); // GPS module using Serial1 for communication
LiquidCrystal_I2C lcd(0x27, 20, 4); // LCD display object (address, size x size)
MFRC522 mfrc522(SS_PIN, RST_PIN); // RFID reader object

void adjustForPST(int &hour, int &day, int &month, int &year);
int getDaysInMonth(int month, int year);
bool isLeapYear(int year);
void displayGPSData(bool gpsFix, int hour, int day, int month, int year, bool
&initialDisplay);
void displayRFIDData(int hour, int day, int month, int year);
void writeSDCard(String sData);
void readSDCard();

void setup() {
  Serial.begin(9600); // Serial communications for debugging
  Serial1.begin(9600); // Serial communications for GPS module

  // Initialize SPI for RFID and RFID module
  SPI.begin();
  mfrc522.PCD_Init();

  // Initialize LCD display
  lcd.init();
  lcd.backlight();

  // Send configuration commands to GPS module
  GPS.sendCommand(PMTK_SET_NMEA_OUTPUT_RMCGGA);
  GPS.sendCommand(PMTK_SET_NMEA_UPDATE_1HZ); // Set GPS update rate to 1Hz
  GPS.sendCommand(PGCMD_ANTENNA); // Request updates on antenna status. Not really
needed for us..

  //Initialize SPI SS, SD library functions will not work if not done
  pinMode(SS, OUTPUT);
  if (!SD.begin(chipSelect, 11, 12, 13)) {
    Serial.println("initialization SD Card failed!");
    return;
  }

  // Debugging
  Serial.println("Initialization/Setup Complete");
}

void loop() {
  // Variables to track GPS data updates and fix status
  bool gpsDataUpdated = false; // Flag to check if new GPS data is available
  bool gpsFix = false; // Flag to check if GPS has a fix
```

```

int hour, day, month, year; // Variables to store GPS time data
static bool initialDisplay = true; // Flag for initial screen setup

// Continuously read data from GPS
while (Serial1.available()) {
    char data = GPS.read(); // Read data from GPS
    if (GPS.newNMEAreceived()) {
        if (GPS.parse(GPS.lastNMEA())) {
            gpsDataUpdated = true; // Flag indicating new GPS data is available
            gpsFix = GPS.fix; // Update GPS fix status

            // Copy GPS data to local variables. Always update time data
            hour = GPS.hour;
            day = GPS.day;
            month = GPS.month;
            year = GPS.year;

            // Adjust the GPS time for PST
            adjustForPST(hour, day, month, year);
        }
    }
}

// Display GPS data on LCD if updated
if (gpsDataUpdated) {
    displayGPSData(gpsFix, hour, day, month, year, initialDisplay);
}

// Check for new RFID card without blocking
if (mfrc522.PICC_IsNewCardPresent() && mfrc522.PICC_ReadCardSerial()) {
    displayRFIDData(hour, day, month, year);
    delay(5000); // Display RFID data for 5 seconds
    initialDisplay = true; // Flag to redraw the screen after RFID display
}
}

// Adjusts GPS time from UTC to PST
void adjustForPST(int &hour, int &day, int &month, int &year) {
    hour -= 8; // Subtract 8 hours for PST
    // Adjust day, month, and year if necessary
    if (hour < 0) {
        hour += 24;
        day--;
        if (day < 1) {
            month--;
            if (month < 1) {
                month = 12;
                year--;
            }
            day = getDaysInMonth(month, year);
        }
    }
}

// Returns the number of days in a given month, accounting for leap years
int getDaysInMonth(int month, int year) {
    int daysInMonth[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
    if (month == 2 && isLeapYear(year)) {
        return 29;
    }
    return daysInMonth[month - 1];
}

// Determines if a year is a leap year
bool isLeapYear(int year) {
    return (year % 4 == 0 && (year % 100 != 0 || year % 400 == 0));
}

```

```

}

// Displays GPS data on the LCD
void displayGPSData(bool gpsFix, int hour, int day, int month, int year, bool
&initialDisplay) {
    static int lastHour = -1, lastDay = -1, lastMonth = -1, lastYear = -1;
    static int lastMinute = -1, lastSecond = -1;

    // Redraw the entire screen if coming back from RFID display message
    if (initialDisplay) {
        lcd.clear(); // Clear the screen

        lcd.setCursor(0, 0);
        lcd.print("Please tap card.    ");

        // Prepare to redraw the date and time
        lcd.setCursor(2, 1);
        lcd.print("Date:          "); // Spaces to clear the line
        lcd.setCursor(2, 2);
        lcd.print("Time:          "); // Spaces to clear the line
        lcd.setCursor(0, 3);
        lcd.print(gpsFix ? "Connected to GPS!    " : "No GPS Fix...    "); // Update
GPS status

        // Reset last known values to force update all data
        lastHour = -1; lastDay = -1; lastMonth = -1; lastYear = -1;
        lastMinute = -1; lastSecond = -1;

        initialDisplay = false;
    }

    // Update the date only if it has changed
    if (lastDay != day || lastMonth != month || lastYear != year) {
        lcd.setCursor(2, 1);
        lcd.print("Date: ");
        lcd.print(month < 10 ? "0" : ""); // Leading zero for month
        lcd.print(month, DEC); lcd.print('/');
        lcd.print(day < 10 ? "0" : ""); // Leading zero for day
        lcd.print(day, DEC); lcd.print('/');
        lcd.print(year % 100); // Display last two digits of the year
        lcd.print("    "); // Add spaces to clear the rest of the line
        lastDay = day; lastMonth = month; lastYear = year;
    }

    // Update the time only if it has changed
    if (lastHour != hour || lastMinute != GPS.minute || lastSecond != GPS.seconds) {
        lcd.setCursor(2, 2);
        lcd.print("Time: ");
        lcd.print(hour < 10 ? "0" : ""); // Leading zero for hours
        lcd.print(hour, DEC); lcd.print(':');
        lcd.print(GPS.minute < 10 ? "0" : ""); // Leading zero for minutes
        lcd.print(GPS.minute, DEC); lcd.print(':');
        lcd.print(GPS.seconds < 10 ? "0" : ""); // Leading zero for seconds
        lcd.print(GPS.seconds, DEC);
        lcd.print("    "); // Add spaces to clear the rest of the line
        lastHour = hour; lastMinute = GPS.minute; lastSecond = GPS.seconds;
    }

    // Update the GPS status only if it has changed
    static bool lastGpsFix = !gpsFix; // Initialize to the opposite of gpsFix to ensure
initial update
    if (lastGpsFix != gpsFix) {
        lcd.setCursor(0, 3);
        lcd.print(gpsFix ? "Connected to GPS!    " : "No GPS Fix...    "); // Extra
spaces to clear previously displayed text
        lastGpsFix = gpsFix;
    }
}

```

```

    }
}

// Displays RFID card data on the LCD
void displayRFIDData(int hour, int day, int month, int year) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Card Detected.");

    // Display the UID of the RFID card
    lcd.setCursor(0, 1);
    lcd.print("UID: ");
    String sPunchclockData = "";
    for (byte i = 0; i < mfrc522.uid.size; i++) {
        lcd.print(mfrc522.uid.uidByte[i] < 0x10 ? "0" : "");
        sPunchclockData = sPunchclockData + String(mfrc522.uid.uidByte[i] < 0x10 ? "0" :
        "");
        lcd.print(mfrc522.uid.uidByte[i], HEX);
        sPunchclockData = sPunchclockData + String(mfrc522.uid.uidByte[i], HEX);
    }

    sPunchclockData.toUpperCase(); // Upper case HEX

    // Display the current date
    lcd.setCursor(2, 2);
    lcd.print("Date: ");
    sPunchclockData = sPunchclockData + String(", ");
    lcd.print(month < 10 ? "0" : ""); // Leading zero for month
    sPunchclockData = sPunchclockData + String(month < 10 ? "0" : "");
    lcd.print(month, DEC);
    sPunchclockData = sPunchclockData + String(month, DEC);
    lcd.print('/');
    sPunchclockData = sPunchclockData + String('/');
    lcd.print(day < 10 ? "0" : ""); // Leading zero for day
    sPunchclockData = sPunchclockData + String(day < 10 ? "0" : "");
    lcd.print(day, DEC);
    sPunchclockData = sPunchclockData + String(day, DEC);
    lcd.print('/');
    sPunchclockData = sPunchclockData + String('/');
    lcd.print(year % 100); // Display last two digits of the year
    sPunchclockData = sPunchclockData + String(year % 100);

    // Display the current time
    lcd.setCursor(2, 3);
    lcd.print("Time: ");
    sPunchclockData = sPunchclockData + String(", ");
    lcd.print(hour < 10 ? "0" : ""); // Leading zero for hours
    sPunchclockData = sPunchclockData + String(hour < 10 ? "0" : "");
    lcd.print(hour, DEC);
    sPunchclockData = sPunchclockData + String(hour, DEC);
    lcd.print(':');
    sPunchclockData = sPunchclockData + String(':');
    lcd.print(GPS.minute < 10 ? "0" : ""); // Leading zero for minutes
    sPunchclockData = sPunchclockData + String(GPS.minute < 10 ? "0" : "");
    lcd.print(GPS.minute, DEC);
    sPunchclockData = sPunchclockData + String(GPS.minute, DEC);
    lcd.print(':');
    sPunchclockData = sPunchclockData + String(':');
    lcd.print(GPS.seconds < 10 ? "0" : ""); // Leading zero for seconds
    sPunchclockData = sPunchclockData + String(GPS.seconds < 10 ? "0" : "");
    lcd.print(GPS.seconds, DEC);
    sPunchclockData = sPunchclockData + String(GPS.seconds, DEC);

    //Serial.println(sPunchclockData);

    writeSDCard(sPunchclockData);
}

```

```

    readSDCard();
}

// Writes RFID UID, Date, and Time to SD Card
void writeSDCard(String sData) {
    // open the file. note that only one file can be open at a time,
    // so you have to close this one before opening another.
    myFile = SD.open(LOG_FILE, FILE_WRITE);

    // if the file opened okay, write to it:
    if (myFile) {
        Serial.print("Writing to Punch.txt...");
        myFile.println(sData);
    // close the file:
        myFile.close();
        Serial.println("done.");
    } else {
        // if the file didn't open, print an error:
        Serial.println("error opening Punch.txt");
    }
}

// Read RFID UID, Date, and Time from SD Card
void readSDCard() {
    // open the file. note that only one file can be open at a time,
    // so you have to close this one before opening another.
    // re-open the file for reading:
    myFile = SD.open(LOG_FILE);
    if (myFile) {
        Serial.println("Punch.txt:");

        // read from the file until there's nothing else in it:
        while (myFile.available()) {
            Serial.write(myFile.read());
        }
        // close the file:
        myFile.close();
    } else {
        // if the file didn't open, print an error:
        Serial.println("error opening Punch.txt");
    }
}
}

```